Lab 1: Intro to ROS 2 and Setting up the RoboRacer Simulator

Learning Goals

Part 1 | Intro to ROS2 :

- Getting familiar with ROS 2 workflow
- Understanding how to create nodes with publishers, subscribers
- Understanding ROS 2 package structure, files, dependencies
- Creating launch files

Part 2 | Setting up the RoboRacer :

- Download and install the RoboRacer Simulator
- Familiarizing yourself with the vehicle model, default ROS topics published and keyboard control.

Before you start

It's highly recommended to install Ubuntu natively on your machine for development in simulation. However, if it's impossible for you to do so, you can still use the simulation inside Docker containers. In the following instructions, if you have Ubuntu installed natively, ignore directions for using Docker.

1. Lab Policies:

This is an individual assignment. You may discuss the assignment with other students; however, you must write and submit your own code. If you discuss the assignment with others, you are required to disclose the names of any students you worked with in the Canvas comments section when submitting your assignment.

Artificial Intelligence Disclosure:
If you used any generative AI tools to complete this assignment, you must disclose that you used them and how in the canvas comment section for the submission.

You may use generative AI in this course for brainstorming ideas, understanding concepts and documentation, debugging assistance, code explanation, refactoring suggestions, and improving clarity or organization of written or technical material.

You may NOT use generative AI to generate complete solutions, core project implementations, or competition code without substantial student contribution and understanding; during exams, in-class evaluations, or project demonstrations, unless explicitly permitted; or by uploading private assignment or project code to external AI services.

Please refer to the course syllabus for the complete policy on AI usage.

2. Overview

The goal of this lab is to get you familiar with the ROS 2 workflow. You'll have the option to complete the coding segment of this assignment in either Python or C++. However, we highly recommend trying out both since this will be the easiest assignment to get started on a new language, and the workflow in these two languages are slightly different in ROS2 and it's beneficial to understand both.

In this lab, it'll be helpful to read these tutorials if you're stuck:

https://docs.ros.org/en/foxy/Tutorials.html

https://roboticsbackend.com/category/ros2/

2.1 Getting ready (Native Ubuntu)

Install ROS 2 following the instructions here:
https://docs.ros.org/en/foxy/Installation.html.

Next, create a workspace:

```shell
mkdir -p ~/lab1_ws/src
cd lab1_ws
colcon build
```

Move on to *Section 3* once you're done.

2.2 Getting ready (Docker)

If you can't have Ubuntu installed natively, install Docker on your system following the instructions here: https://docs.docker.com/get-docker/. The documentation of Docker can be found here.

Next, start a container with a bind mount to your workspace directory on your host system inside this repo by:

```shell
docker run -it -v
<absolute_path_to_this_repo>/lab1_ws/src/:/lab1_ws/src/
--name f1tenth_lab1 ros:foxy
```

This will create a workspace directory on the host at `<absolute_path_to_this_repo>/lab1_ws/src`. It'll create the container based on the official ROS 2 Foxy image, and give the container a name `f1tenth_lab1`. You'll then have access to a terminal inside the container.

`tmux` is recommended when you're working inside a container. It could be installed in the container via: `apt update && apt install tmux`. `tmux` allows you to have multiple `bash` session in the same terminal window. This will be very convenient working inside containers. A quick reference on how to use tmux can be found here. You can start a

session with `tmux`. Then you can call different `tmux` commands by pressing `ctrl+B` first and then the corresponding key. For example, to add a new window, press `ctrl+B` first and release and press `c` to create a new window. You can also move around with `ctrl+B` then `n` or `p`.

A cheatsheet for the original tmux shortcut keys can be found here. To know about how to change the configuration of tmux to make it more useable (for example, if you want to toggle the mouse mode on when you start a tmux bash session or change the shortcut keys), you can find a tutorial here.

3: Part 1: ROS 2 Basics

Now that we have the access to a ROS 2 environment, let's test out the basic ROS 2 commands. In the terminal, run:

```shell
source /opt/ros/foxy/setup.bash
ros2 topic list
```

You should see two topics listed:

```shell
/parameter_events
/rosout
```

If you need multiple terminals and you're inside a Docker container, use `tmux`.

3.1: Creating a Package

Task 1: create a package named `lab1_pkg` in the workspace we created. The package needs to meet these criteria:

- The package supports both `Python` and `C++`.
- The package needs to have the `ackermann_msgs` dependency.
- Both of these can be done by declaring the correct dependencies in `package.xml`.
- If declared properly the depencies could be installed using `rosdep`.
- Your package folder should be neat. You shouldn't have multiple 'src' folders or unnecessary 'install' or 'build' folders.

3.2: Creating nodes with publishers and subscribers

Task 2: create two nodes in the package we just created. You can use either `Python` or `C++` for these nodes.

The first node will be named `talker.cpp` or `talker.py` and needs to meet these criteria:

- `talker` listens to two ROS parameters `v` and `d`.
- `talker` publishes an `AckermannDriveStamped` message with the `speed` field equal to the `v` parameter and `steering_angle` field equal to the `d` parameter, and to a topic named `drive`.
- `talker` publishes as fast as possible.
- To test node, set the two ROS parameters through command line, a launch file, or a yaml file.

The second node will be named `relay.cpp` or `relay.py` and needs to meet these criteria:

- `relay` subscribes to the `drive` topic.
- In the subscriber callback, take the speed and steering angle from the incoming message, multiply both by 3, and publish the new values via another `AckermannDriveStamped` message to a topic named `drive_relay`.

3.3: Creating a launch file and a parameter file

Task 3: create a launch file `lab1_launch.py` that launches both of the nodes we've created. If you want, you could also set the parameter for the `talker` node in this launch file.

3.4: ROS 2 commands

After you've finished all the deliverables, launch the two nodes and test out these ROS 2 commands:

```shell
ros2 topic list
ros2 topic info drive
ros2 topic echo drive
ros2 node list
ros2 node info talker
ros2 node info relay
```

3.5: Deliverables and Submission

Deliverable: To submit this lab, submit a short video of your terminal after executing the commands above. Additionally, upload the node code to the canvas page as a file attachment. A portion of the submissions will be peer reviewed by course staff to ensure that the code is academically honest(i.e. video is not faked, code is implemented properly).

Part 2: Simulator Setup

(not graded but necessary for future labs) :

Alternative Resource:

If you get stuck or prefer video formats for learning please follow this instructional video made by the RoboRacer Community:

Video

F1TENTH gym environment ROS2 communication bridge

This is a containerized ROS communication bridge for the F1TENTH gym environment that turns it into a simulation in ROS2.

Installation

Supported System:

- Ubuntu (tested on 20.04) native with ROS 2
- Ubuntu (tested on 20.04) with an NVIDIA gpu and nvidia-docker2 support
- Windows 10, macOS, and Ubuntu without an NVIDIA gpu (using noVNC)

This installation guide will be split into instruction for installing the ROS 2 package natively, and for systems with or without an NVIDIA gpu in Docker containers.

Native on Ubuntu 20.04

Install the following dependencies:

- **ROS 2** Follow the instructions here to install ROS 2 Foxy.
- **F1TENTH Gym**

```
Shell
git clone https://github.com/f1tenth/f1tenth_gym
cd f1tenth_gym && pip3 install -e .
```

Installing the simulation:

- Create a workspace: `cd $HOME && mkdir -p sim_ws/src`
- Clone the repo into the workspace:

```
Shell
cd $HOME/sim_ws/src
git clone https://github.com/f1tenth/f1tenth_gym_ros
```

- Update correct parameter for path to map file: Go to `sim.yaml` https://github.com/f1tenth/f1tenth_gym_ros/blob/main/config/sim.yaml in your cloned repo, change the `map_path` parameter to point to the correct location. It should be `'<your_home_dir>/sim_ws/src/f1tenth_gym_ros/maps/levine'`
- Install dependencies with rosdep:

```
Shell
source /opt/ros/foxy/setup.bash
cd ..
rosdep install -i --from-path src --rosdistro foxy -y
```

- Build the workspace: `colcon build`

With an NVIDIA gpu:

Install the following dependencies:

- **Docker** Follow the instructions here to install Docker. A short tutorial can be found here if you're not familiar with Docker. If you followed the post-installation steps you won't have to prepend your docker and docker-compose commands with sudo.
- **nvidia-docker2**, follow the instructions here if you have a support GPU. It is also possible to use Intel integrated graphics to forward the display, see details instructions from the Rocker repo. If you are on windows with an NVIDIA GPU, you'll have to use WSL (Windows Subsystem for Linux). Please refer to the guide here, here, and here.
- **rocker** https://github.com/osrf/rocker. This is a tool developed by OSRF to run Docker images with local support injected. We use it for GUI forwarding. If you're on Windows, WSL should also support this.

Installing the simulation:

1. Clone this repo
2. Build the docker image by:

```
Shell
$ cd f1tenth_gym_ros
$ docker build -t f1tenth_gym_ros -f Dockerfile .
```

3. To run the containerized environment, start a docker container by running the following. (example showned here with nvidia-docker support). By running this, the current directory that you're in (should be `f1tenth_gym_ros`) is mounted in the container at `/sim_ws/src/f1tenth_gym_ros`. Which means that the changes you make in the repo on the host system will also reflect in the container.

```
Shell
$ rocker --nvidia --x11 --volume .:/sim_ws/src/f1tenth_gym_ros --
f1tenth_gym_ros
```

Without an NVIDIA gpu:

Install the following dependencies:

If your system does not support nvidia-docker2, noVNC will have to be used to forward the display.

- Again you'll need **Docker**. Follow the instruction from above.
- Additionally you'll need **docker-compose**. Follow the instruction here to install docker-compose.

Installing the simulation:

1. Clone this repo
2. Bringup the novnc container and the sim container with docker-compose:

```
Shell
docker-compose up
```

3. In a separate terminal, run the following, and you'll have the a bash session in the simulation container. `tmux` is available for convenience.

```
Shell
docker exec -it f1tenth_gym_ros-sim-1 /bin/bash
```

4. In your browser, navigate to http://localhost:8080/vnc.html, you should see the noVNC logo with the connect button. Click the connect button to connect to the session.

Launching the Simulation

1. `tmux` is included in the contianer, so you can create multiple bash sessions in the same terminal.
2. To launch the simulation, make sure you source both the ROS2 setup script and the local workspace setup script. Run the following in the bash session from the container:

```
Shell
$ source /opt/ros/foxy/setup.bash
$ source install/local_setup.bash
$ ros2 launch f1tenth_gym_ros gym_bridge_launch.py
```

A rviz window should pop up showing the simulation either on your host system or in the browser window depending on the display forwarding you chose.

You can then run another node by creating another bash session in `tmux`.

Configuring the simulation

- The configuration file for the simulation is at `f1tenth_gym_ros/config/sim.yaml`.
- Topic names and namespaces can be configured but is recommended to leave uncahnged.
- The map can be changed via the `map_path` parameter. You'll have to use the full path to the map file in the container. The map follows the ROS convention. It is assumed that the image file and the `yaml` file for the map are in the same directory with the same name. See the note below about mounting a volume to see where to put your map file.
- The `num_agent` parameter can be changed to either 1 or 2 for single or two agent racing.
- The ego and opponent starting pose can also be changed via parameters, these are in the global map coordinate frame.

The entire directory of the repo is mounted to a workspace `/sim_ws/src` as a package. All changes made in the repo on the host system will also reflect in the container. After changing the configuration, run `colcon build` again in the container workspace to make sure the changes are reflected.

Topics published by the simulation

In **single** agent:

`/scan`: The ego agent's laser scan

`/ego_racecar/odom`: The ego agent's odometry

`/map`: The map of the environment

A `tf` tree is also maintained.

In **two** agents:

In addition to the topics available in the single agent scenario, these topics are also available:

`/opp_scan`: The opponent agent's laser scan

`/ego_racecar/opp_odom`: The opponent agent's odometry for the ego agent's planner

`/opp_racecar/odom`: The opponent agents' odometry

`/opp_racecar/opp_odom`: The ego agent's odometry for the opponent agent's planner

Topics subscribed by the simulation

In **single** agent:

`/drive`: The ego agent's drive command via `AckermannDriveStamped` messages

`/initalpose`: This is the topic for resetting the ego's pose via RViz's 2D Pose Estimate tool. Do **NOT** publish directly to this topic unless you know what you're doing.

TODO: kb teleop topics

In **two** agents:

In addition to all topics in the single agent scenario, these topics are also available:

`/opp_drive`: The opponent agent's drive command via `AckermannDriveStamped` messages. Note that you'll need to publish to **both** the ego's drive topic and the opponent's drive topic for the cars to move when using 2 agents.

`/goal_pose`: This is the topic for resetting the opponent agent's pose via RViz's 2D Goal Pose tool. Do **NOT** publish directly to this topic unless you know what you're doing.

Keyboard Teleop

The keyboard teleop node from `teleop_twist_keyboard` is also installed as part of the simulation's dependency. To enable keyboard teleop, set `kb_teleop` to `True` in `sim.yaml`. After launching the simulation, in another terminal, run:

```
Shell
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Then, press `i` to move forward, `u` and `o` to move forward and turn, `,` to move backwards, `m` and `.` to move backwards and turn, and `k` to stop in the terminal window running the teleop node.

Developing and creating your own agent in ROS 2

There are multiple ways to launch your own agent to control the vehicles.

- The first one is creating a new package for your agent in the `/sim_ws` workspace inside the sim container. After launch the simulation, launch the agent node in another bash session while the sim is running.
- The second one is to create a new ROS 2 container for you agent node. Then create your own package and nodes inside. Launch the sim container and the agent container both. With default networking configurations for `docker`, the behavior is to put The two containers on the same network, and they should be able to discover and talk to each other on different topics. If you're using noVNC, create a new service in `docker-compose.yml` for your agent node. You'll also have to put your container on the same network as the sim and novnc containers.